LOW-RANK NETWORKS: LOSS LANDSCAPE VALLEY–FREQUENCY CORRESPONDENCE

Anonymous authors
Paper under double-blind review

ABSTRACT

We study the **relation between feature learning and the loss landscape** for low-rank neural networks trained on frequency-dependent, 1D regression targets. In controlled experiments (matrix-rank nets, sum-of-cosines target, adaptive LR schedule), we observe that: (1) the loss exhibits a *plateau–sharp–plateau* structure where each plateau (or saddle region) corresponds to a frequency band to be learned and the dynamics are *saddle-to-saddle*, so each sharp transition coincides with the network capturing a new frequency and with more oscillatory partial functions (e.g. layer L developing on the order of $2L$ spikes, not observed in standard MLPs); (2) the landscape appears as a *plateau with many sharp, deep basins* that are hard to enter unless the learning rate is small—at each LR divide the loss drops sharply and the trajectory descends into such a basin, with visible refinement of internal representations (we report loss, fit, and partial functions before/after LR divides); (3) scaling of plateau-escape time with learning rate and batch size, and sensitivity of the lower landscape to sample size N, are consistent with the picture that the landscape is aligned with hierarchical feature acquisition. We provide a clear methodology and reproducible configs. Scope: 1D regression and the sumcos/MMNN setting. This submission fits the workshop's focus on empirical studies of loss landscapes and learned representations.

1 INTRODUCTION

A central question in deep learning is how the **geometry of the loss landscape** (plateaus, sharp transitions, basins) connects to **what the network learns** (channel-wise representations, frequency bands, oscillatory structure). For low-rank neural networks, this connection is still poorly understood. In this paper we provide a focused empirical study of the **relation between feature learning and the loss landscape** in such networks, in a setting chosen for controllability and reproducibility.

We use matrix-rank neural networks (MMNNs) and a 1D frequency-dependent target (sum of cosines) so that both the loss curve and the evolution of partial functions (channel outputs) can be measured and compared. Our main observations are: (i) the loss exhibits a plateau–sharp–plateau structure where each plateau (or saddle) corresponds to a frequency to be learned and the dynamics are saddle-to-saddle, so each sharp drop coincides with the acquisition of a new frequency and with more oscillatory partial functions; (ii) the landscape behaves like a plateau with many sharp, deep basins that are hard to reach unless the learning rate is small—at each LR divide the loss drops sharply and we show loss, fit, and partial functions before and after LR divides to make this link explicit; (iii) observables such as the number of minima per channel and the $2L$-spike pattern per layer L provide a concrete bridge between landscape and feature learning. We give a clear methodology (Section 2–3), including the rationale for 1D data and full experimental protocol, and we provide config files and script paths so that results can be reproduced or extended. **Scope and limitations:** the study is restricted to 1D regression and the sumcos/MMNN setting; we do not claim that the same relation holds in higher dimensions or for other architectures without further experiments. The takeaway for the reviewer is that *in this setting*, understanding the loss landscape amounts to understanding how and when features are acquired, and that the relation is testable via the provided methodology.

2 ARCHITECTURE AND TARGET

2.1 TARGET AND DATA

The target is $y(x) = \sum_{k=1}^{\text{factor}} \cos(2\pi k x)$ on $x \in [-1, 1]$ (`target_baseline` in `run_scaling_law_depth_width.py`). For **factor** $= 3$ (f_3), $y(x) = \cos(2\pi x) + \cos(4\pi x) + \cos(6\pi x)$. Training inputs are a uniform grid of N_{train} points; for the reported figures we use $N_{\text{train}} = 768$ and batch size 8. Loss is MSE. In broader baseline sweeps (factor 1–5, $N = \text{base} \times \text{factor}$, ranks 5 and 10, batch sizes 1–16; see `BASELINE_SWEEP_SETUPS_AND_RANKS.md` for batch-size analysis), "worked" configs (test error < 0.01) are 73 for rank 10 and 76 for rank 5 out of 429 and 750 completed runs respectively. **Main conclusion from the full run:** small batch (1, 2, 4) consistently outperforms large batch (8, 16); for rank 5, small bs is optimal and depth $L = 3$ leads to better results than $L = 2$ in the regimes tested. Small batch always performs better in these sweeps, consistent with plateau escape and basin access depending on gradient noise and LR.

2.2 MMNN ARCHITECTURE

The model is a **matrix-rank neural network (MMNN)**: input and output dimension 1; hidden layers have **rank** R and **width** W. For depth L, the layer sequence is $1 \to R \to W \to R \to W \to \cdots \to R \to 1$: each "layer" is (i) a linear map rank→width, (ii) ReLU, (iii) a linear map width→rank, so there are L such blocks. In the reported configs, $W = 1024$, $R = 10$, and $L \in \{2, 3\}$. With **fixWb = True**, the first linear (rank→width) in each block is *frozen*; only the width→rank linears and their biases are trained (low-rank channel weights). Initialization is μ-parameterization: weights uniform in $[-1, 1]$ then scaled by $1/\sqrt{\text{rank}}$ or $1/\sqrt{\text{width}}$ as appropriate. Optimizer is SGD with momentum 0 and gradient clipping (max norm 1.0).

3 METHODOLOGY

3.1 WHY 1D DATA

We use **1-dimensional** regression for three reasons that are standard in mean-field and optimization analyses of low-rank networks. **(i) Mean-field parameterization and convergence of $\mathbb{E}_Z[\cdot]$:** the training dynamics in the mean-field width limit are expressed in terms of expectations over the data distribution, $\mathbb{E}_Z[\cdot]$. With 1D input we can use *large sample sizes* (e.g. $N_{\text{train}} = 768$ or several thousand) so that empirical averages over the training set approximate these expectations well; optimization dynamics are then not confounded by overfitting or sample-complexity effects, and the finite-width behavior is close to the mean-field limit. **(ii) Controlled hierarchical frequency learning:** a frequency-dependent target on $[-1, 1]$ (e.g. sum of cosines) provides a controlled setting to study how the loss landscape and feature learning interact across frequency bands—with a direct link to tractable toy models (e.g. two-point support) and to the interpretation that each sharp loss drop corresponds to capturing a new frequency. **(iii) Interpretability:** 1D allows clear visualization and quantification of channel specialization (spatial location and frequency) and of observables such as the log-ratio evolution and the number of minima per partial-function component.

3.2 TRAINING PROTOCOL AND OBSERVABLES

The initial learning rate is 10^{-2}. The schedule is **adaptive stagnation**: over a sliding window of 10 epochs, if the mean training loss does not improve compared to the previous window, and at least 20 epochs have passed since the last reduction, the learning rate is replaced by the next value in a precomputed sequence (divide by 2 each step, 25 steps; training stops if LR falls below 10^{-6}). At each LR reduction, the script saves (i) the model state *just before* the divide and (ii) the state *10 epochs after* the divide. The reported figures compare these checkpoints: **loss** (training curve), **fit** (prediction vs. target), and **partial functions** (channel outputs at selected layers). Runs are produced by `experiments/table/run_selected_sumcos_configs.py`; configs are in `results_sumcos_selected_rerun/f3_N768_bs8_L2/config.json` (and similarly for $L = 3$). Max epochs 10,000; seed 42. See Appendix for full parameterization.

4 PLATEAU–SHARP–PLATEAU AND FREQUENCY STRUCTURE

In our experiments, the training loss consistently exhibits long *plateaus* interrupted by *sharp* drops. We interpret this as follows: each plateau (or saddle region) corresponds to a *frequency band* that has not yet been learned; the dynamics move *saddle-to-saddle*, so the training trajectory is directly tied to *frequency learning*—each sharp transition marks the acquisition of a new frequency. Initially the predictor is nearly constant and the loss is flat; after an epoch count that depends strongly on the learning rate, a first sharp transition occurs—for lr 10^{-2} we observe a plateau of on the order of **20 epochs** before the first drop, versus hundreds to thousands of epochs for lr 10^{-4}. **Feature–landscape link:** we observe that each such transition coincides with the network capturing a new frequency band and with the internal partial functions (channel outputs) gaining more oscillatory structure—e.g. an increase in the mean number of strict local minima per component in deeper layers. The landscape thus displays a hierarchical structure aligned with hierarchical feature learning: low-to-high frequency in the target is reflected in staged refinement of the learned representation.

Early in training, many parameter directions give similar loss (landscape appears flat in those directions). Low-rank parameterizations reduce the number of effective directions, which we hypothesize makes the landscape more structured and interpretable in terms of the r channels. After plateaus, the dynamics can be described as entering a favorable basin; learning-rate reduction at the right time triggers this descent and, with it, a visible change in the partial functions (Section 8).

5 LEARNING RATE AND SHARP BASINS ON A PLATEAU

The loss drops *sharply and almost instantaneously* at each learning-rate (LR) divide in our runs (Figures 1 and 2). This suggests that the landscape has **very sharp basins**—deep, narrow holes—sitting on top of a plateau: the trajectory wanders on the plateau until LR is reduced, at which point it can descend into one of these holes and the loss falls abruptly. Thus the picture is a *plateau with many sharp holes that are hard to enter unless the learning rate is small*; with large LR the optimizer overshoots or fails to settle into these basins. **Feature–landscape link:** entering such a basin is accompanied by a visible refinement of the learned features: the partial functions before vs. after an LR divide (Figure 3) show how internal representations change when the trajectory drops into a sharper basin. After long plateaus, the landscape appears to contain multiple such basins that are not accessible in the initial phase; a learning rate that is too large may prevent the trajectory from entering them and thus from reaching the feature configurations that support low loss. This is consistent with *grokking*-like phenomena, where generalization improves after prolonged training once the dynamics enter a favorable basin and the corresponding feature structure is acquired.

6 ADAM, EOS JUMP, AND INSTABILITY

In our setting, Adam training is observed to be highly unstable. A notable empirical finding is that a *jump* in the trajectory near *end-of-schedule* (EOS) can land the solution in a sharp but small basin—in some runs with a shape resembling multiple local minima and the global minimum in the middle. When such a jump occurs at EOS, the optimizer effectively reaches a favorable basin; this may partly explain why Adam can perform well in practice despite instability. The *lower part* of the landscape (near good minima) appears more sensitive to the number of samples N than the upper part; under unstable or very long training, behavior can exhibit unexpected sensitivity or non-monotonicity. When instability is present, training curves sometimes display a characteristic symmetry; we leave a systematic characterization for future work.

7 SCALING AND THE FEATURE–LANDSCAPE RELATION

Scaling dimensions that affect both the loss landscape and feature learning in our experiments include the following. **Depth and width:** we observe that layer L can exhibit on the order of $2L$ spikes (or similar oscillatory structure) in the partial functions, a pattern not observed in standard MLPs and thus a distinctive feature-learning signature of low-rank architectures that is tied to the hierarchical loss landscape (deeper layers $\Rightarrow$ more complex features $\Rightarrow$ later sharp transitions). **Number of samples N:** in line with mean-field considerations, width on the order of N_{train} or larger helps

approximate expectations; the lower part of the landscape (and the corresponding refined features) can be sensitive to N. **Rank r:** rank can have a strong beneficial effect in some regimes, and the r channels organize how features are distributed across the landscape. **Frequency and learning rate:** hierarchical Fourier targets (low then high frequency) lead to better learning than high frequency alone in our runs; smaller learning rates allow the trajectory to enter basins where high-frequency features are acquired.

We observe that **plateau-escape time** (epoch to leave the first plateau) scales roughly like $1/\mathrm{lr}$ (e.g. on the order of 20 epochs for lr 10^{-2} and $\sim$ 2000 for lr 10^{-4}), with additional dependence on batch size (e.g. through gradient variance, suggesting a $\sqrt{\mathrm{batch}}$ effect). There is an *optimal batch size* in this setting: small batch (1–4) escapes plateaus very well, especially for large factor and low rank ($r = 5$); baseline sweeps (sumcos, batch 1–16, ranks 5, 10, 20) consistently show that small batch trains better than large (8, 16) in terms of worked configs and best test error (see `INDEX_LOGRATIO_ESCAPE_SCALING.md`, `EXHAUSTIVE_RESULTS_BASELINE_SWEEP_RANKS.md`). Figures **??** and **??** show the empirical **epochs to escape** vs. learning rate and vs. batch size from plateau-escape runs (`experiments/table/plateau/`). In baseline sweeps, best configs for $r = 5$, factor 4–5, use small bs and $L = 3$ (e.g. f4_N1024_bs4_L3, f5_N1280_bs4_L3 with test error $\sim 1.6\times10^{-3}$ and $\sim 3.3\times10^{-3}$); SGD with larger batch (8, 16) gives fewer worked runs and worse best error. In separate frequency/layer-scaling experiments (multi-frequency target, Adam, StepLR, 58 configs), low frequency multipliers (0.3, 0.6) achieve test error $\sim 10^{-6}$–10^{-5} with moderate depth, whereas higher frequency (1.5 and above) fails to converge (test error ~ 1 or more); very deep nets (e.g. $L = 56$, $L = 80$) can exhibit blow-up or NaN, indicating a *stability frontier* for depth in this setting. In some runs the landscape appears to exhibit self-similarity (training curves resembling superposed segments), suggesting corresponding structure in feature acquisition.

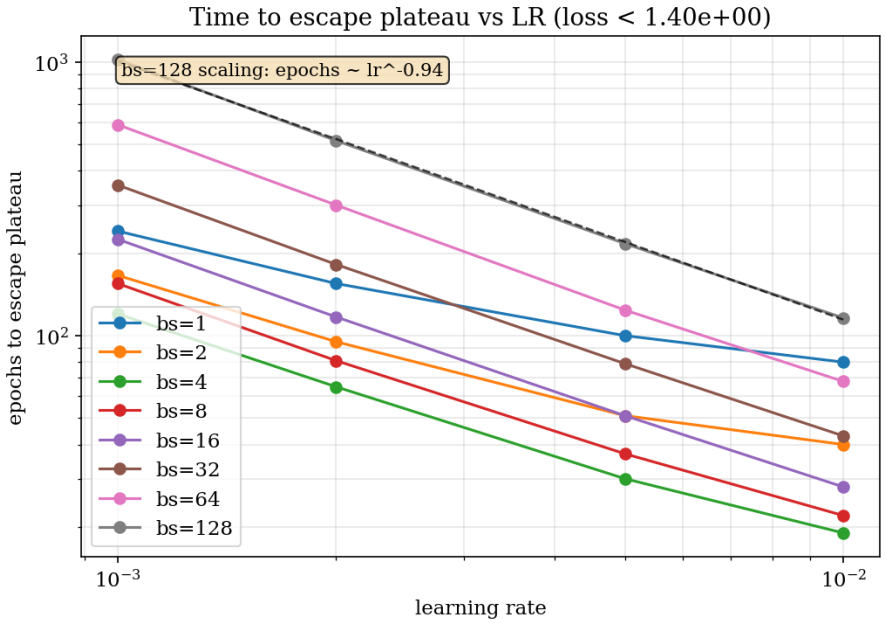


Figure 1: Epochs to escape the first plateau vs. learning rate (plateau-escape runs). Consistent with scaling $\propto 1/\mathrm{lr}$.

8 Figures: Loss, Fit, and Partials Before/After LR Reduction

We illustrate the **feature–landscape relation** with the runs described in Sections 2 and 3 (factor f_3, $N = 768$, batch size 8, depths $L = 2$ and $L = 3$). Figure 1 and Figure 2 show loss and fit before and after LR divides for $L = 2$ and $L = 3$. The *sharp, nearly instantaneous* loss drops at each LR divide (visible in the curves) are the empirical signature of the picture in Section 5: the landscape

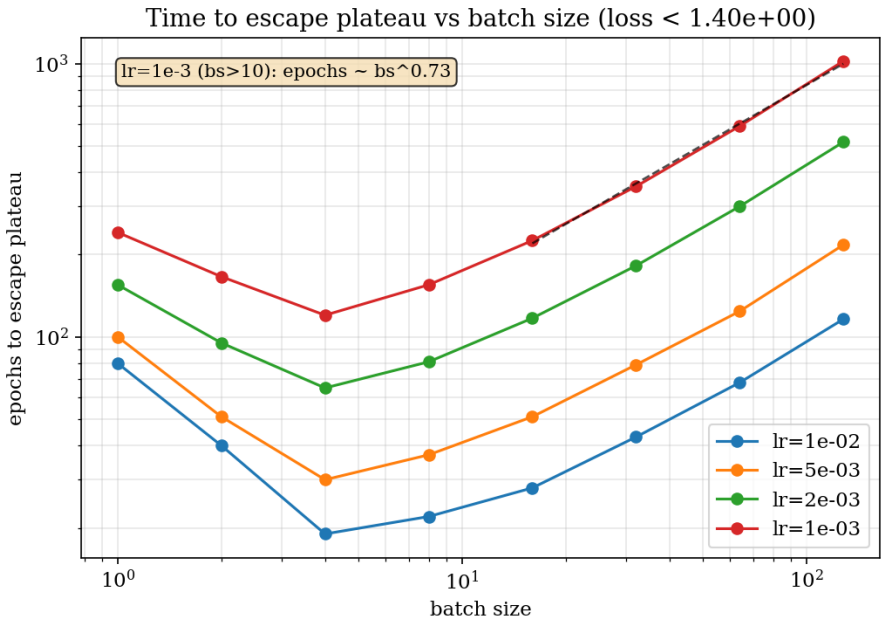


Figure 2: Epochs to escape vs. batch size. Small batch (1–4) escapes effectively; optimal/critical batch size in this setting.

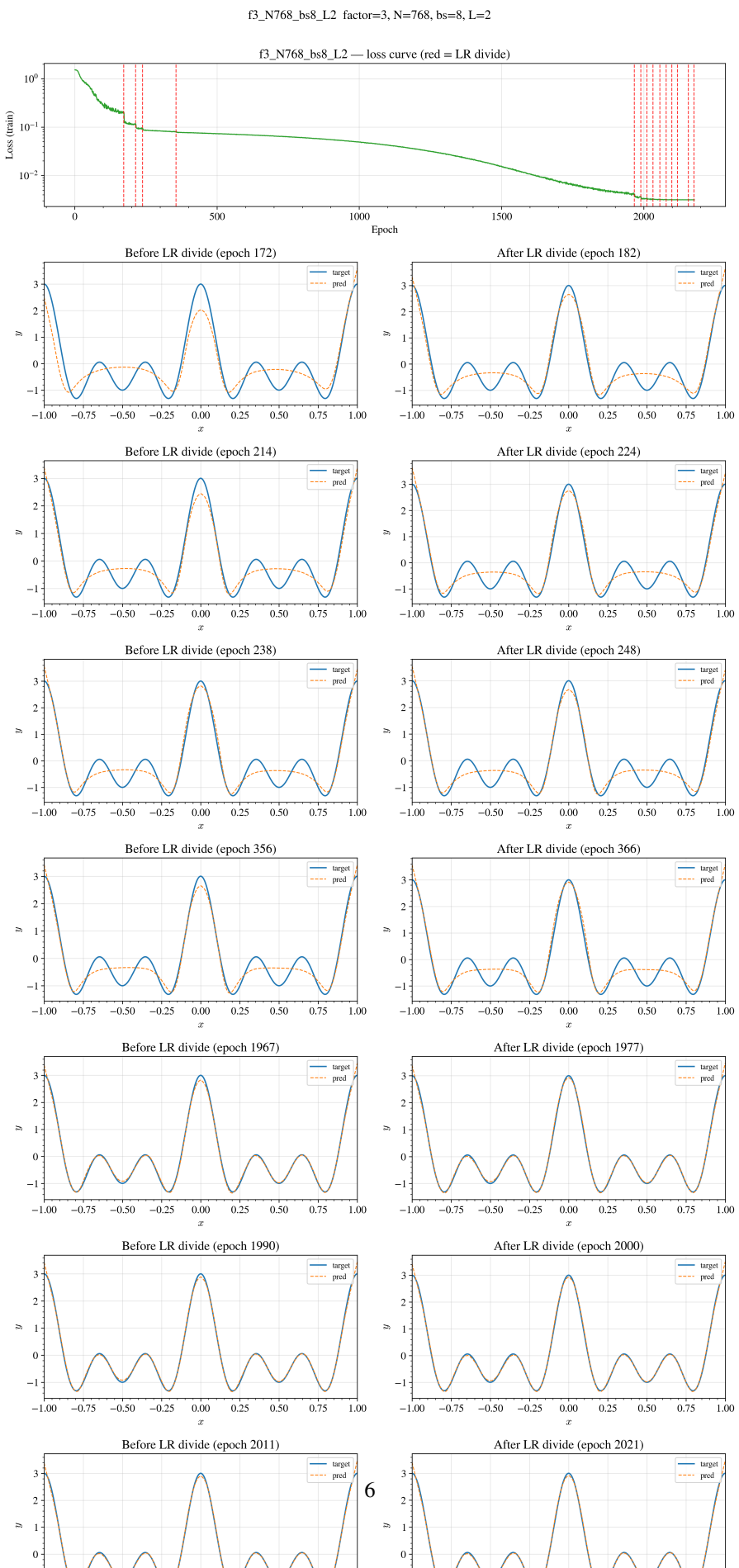
is a plateau with sharp, deep basins that become accessible when LR is reduced. The quality of the fit improves in step with these drops. Figure 3 shows the evolution of *partial functions* (channel outputs) before and after LR divides for $L = 3$—the internal feature representation changes when the trajectory enters a sharper basin, making the link between landscape and feature learning directly visible. **Log-ratio trajectories** (channel dominance over training) further illustrate how LR and landscape shape specialization: at lr 2×10^{-3}, bs 4, the log-ratio trajectories exhibit *divergence* globally by the end of training (Figure 4); at lr 2×10^{-4}, bs 4, they exhibit *convergence* (Figure 5). So the same architecture can either specialize (convergent log-ratios) or fail to stabilize (divergent) depending on the training regime, consistent with the valley–frequency picture. Channel-wise feature learning (log-ratio dynamics, spike specialization) is analyzed in a companion SciForDL submission (`features.tex`).

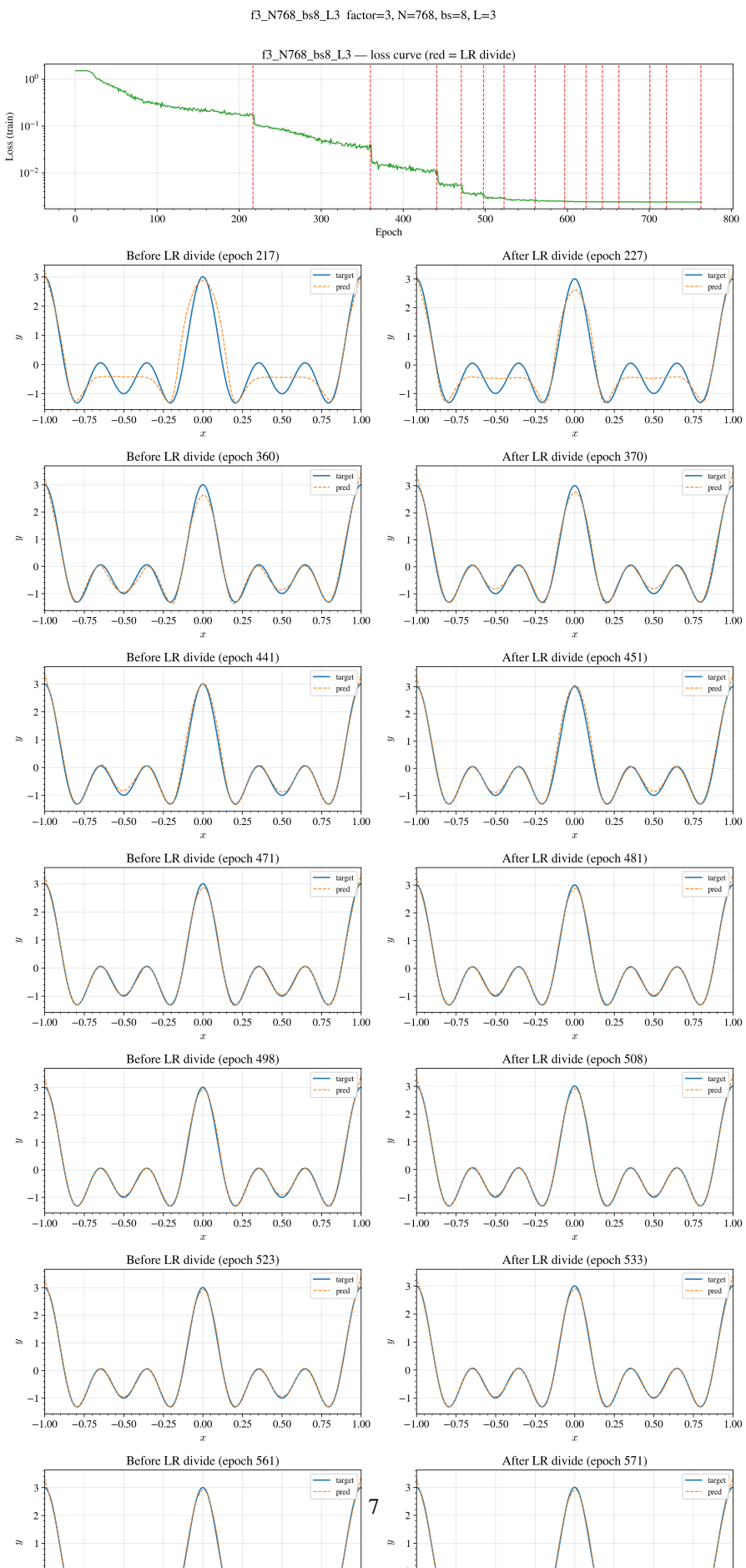
9 Limitations

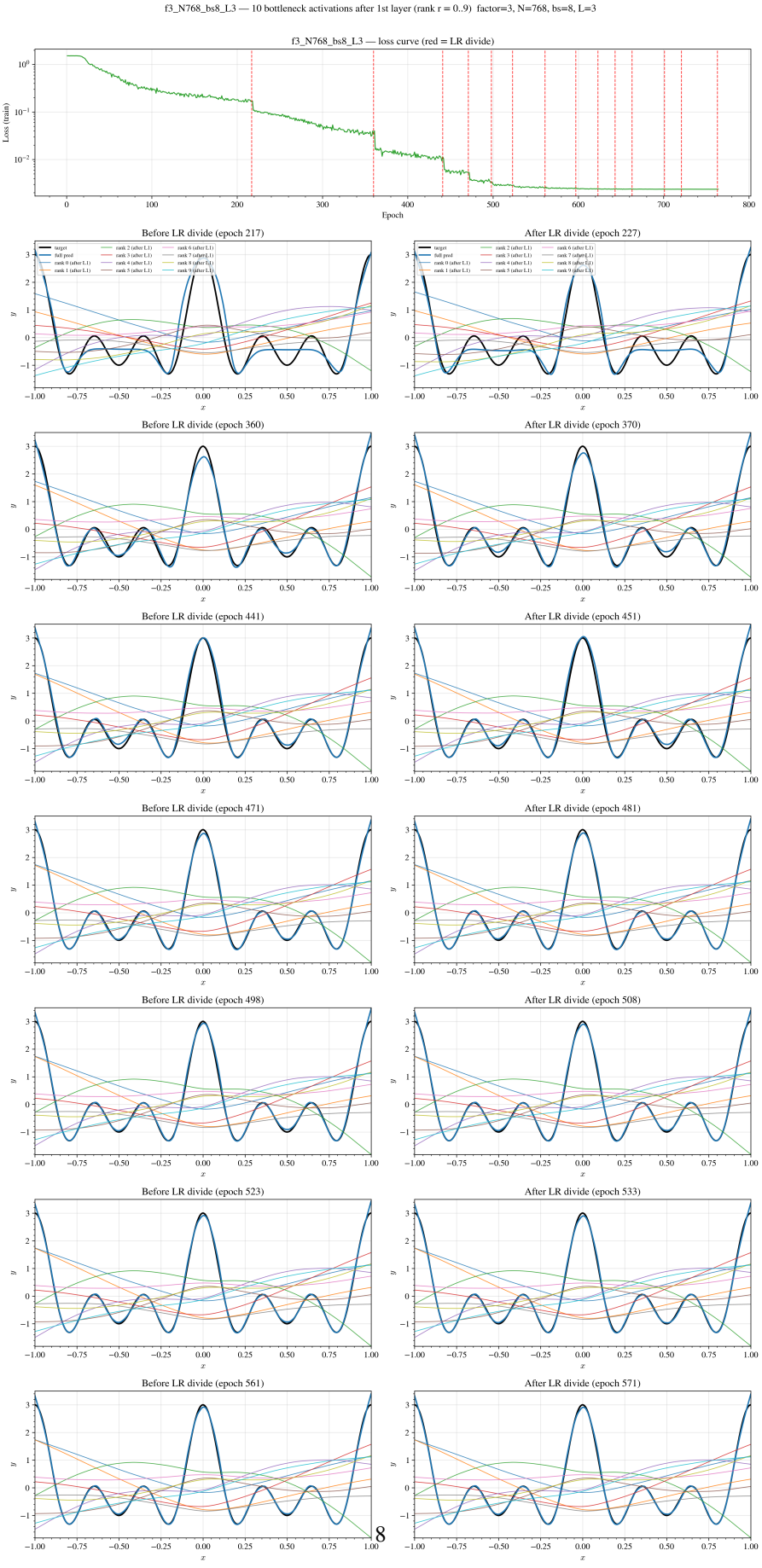
This study is restricted to 1D regression with the sum-of-cosines target and the MMNN architecture; we do not claim that the same feature–landscape relation holds in higher dimensions or for other architectures without further experiments. Observations on Adam (EOS jump, instability) and on self-similarity of training curves are qualitative; systematic replication over seeds and hyperparameters would be needed to establish robust scaling statements. In frequency/layer-scaling runs, very deep networks ($L = 56$, $L = 80$) can become unstable (huge test error or NaN), so depth scaling has a stability frontier in the current setup. The reported configs and script paths (including `run_scaling_law_depth_width.py`, `analyze_baseline_sweep.py`) are intended to make the methodology reproducible so that the relation can be verified or refuted.

10 Conclusion

We described the **relation between feature learning and the loss landscape** for low-rank neural networks in a controlled 1D setting. The two are tightly linked: (i) each plateau (or saddle) corresponds to a frequency to be learned and the dynamics are saddle-to-saddle, so the plateau–sharp–plateau loss structure corresponds to staged acquisition of frequency bands and to more oscillatory partial functions (e.g. mean minima per component increasing with depth); (ii) the landscape is


f3_N768_bs8_L2 factor=3, N=768, bs=8, L=2
f3_N768_bs8_L2 — loss curve (red = LR divide)
Loss (train)
Epoch
Before LR divide (epoch 172)
After LR divide (epoch 182)
Before LR divide (epoch 214)
After LR divide (epoch 224)
Before LR divide (epoch 238)
After LR divide (epoch 248)
Before LR divide (epoch 356)
After LR divide (epoch 366)
Before LR divide (epoch 1967)
After LR divide (epoch 1977)
Before LR divide (epoch 1990)
After LR divide (epoch 2000)
Before LR divide (epoch 2011)
After LR divide (epoch 2021)
target
pred
x
y


f3_N768_bs8_L3 factor=3, N=768, bs=8, L=3
f3_N768_bs8_L3 — loss curve (red = LR divide)
Loss (train)
Epoch
Before LR divide (epoch 217)
After LR divide (epoch 227)
Before LR divide (epoch 360)
After LR divide (epoch 370)
Before LR divide (epoch 441)
After LR divide (epoch 451)
Before LR divide (epoch 471)
After LR divide (epoch 481)
Before LR divide (epoch 498)
After LR divide (epoch 508)
Before LR divide (epoch 523)
After LR divide (epoch 533)
Before LR divide (epoch 561)
After LR divide (epoch 571)
target
pred
x
y


f3_N768_bs8_L3 — 10 bottleneck activations after 1st layer (rank r = 0..9) factor=3, N=768, bs=8, L=3
f3_N768_bs8_L3 — loss curve (red = LR divide)
Loss (train)
Epoch
Before LR divide (epoch 217)
After LR divide (epoch 227)
Before LR divide (epoch 360)
After LR divide (epoch 370)
Before LR divide (epoch 441)
After LR divide (epoch 451)
Before LR divide (epoch 471)
After LR divide (epoch 481)
Before LR divide (epoch 498)
After LR divide (epoch 508)
Before LR divide (epoch 523)
After LR divide (epoch 533)
Before LR divide (epoch 561)
After LR divide (epoch 571)
Before LR divide (epoch 597)
After LR divide (epoch 607)

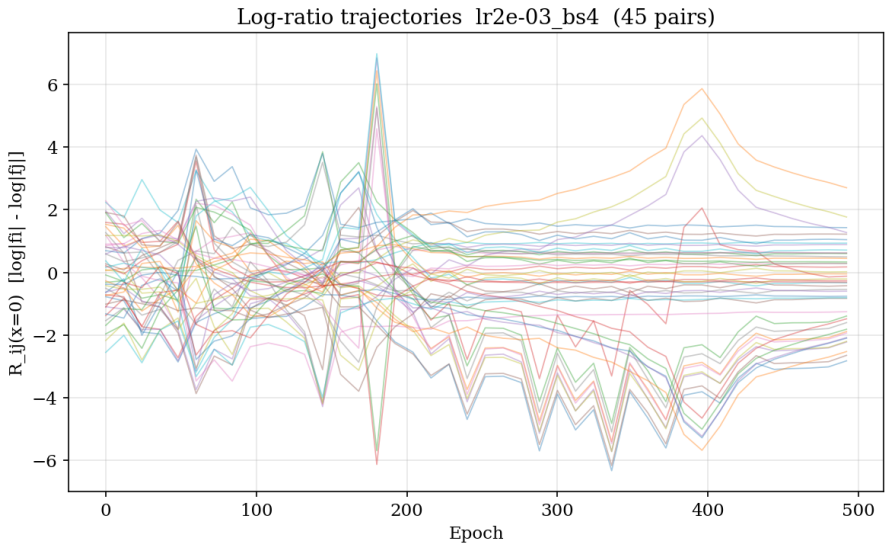


Figure 6: Log-ratio trajectories (channel dominance over training); lr 2×10^{-3}, bs 4. Trajectories diverge globally by end of training.

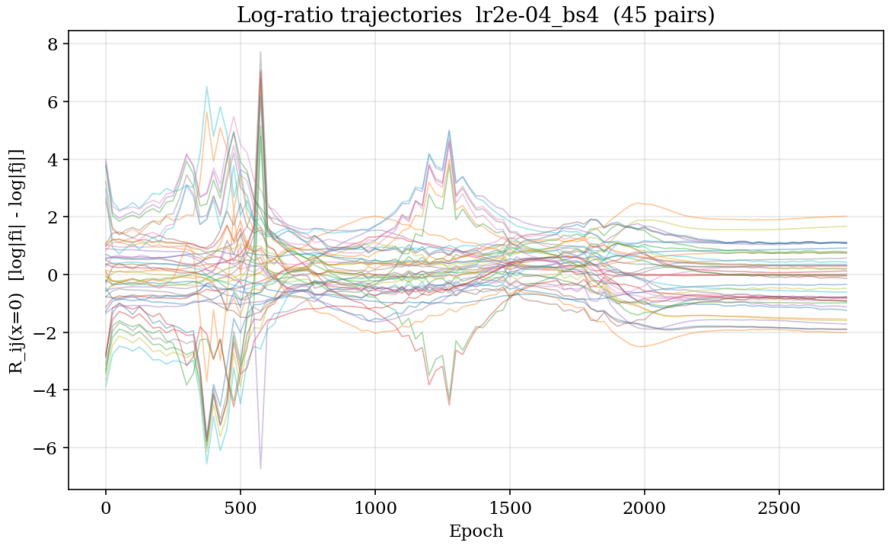


Figure 7: Log-ratio trajectories; lr 2×10^{-4}, bs 4. Trajectories converge by end of training (stable specialization).

a plateau with sharp, deep basins that are hard to enter unless the learning rate is small—at each LR divide the loss drops sharply and the trajectory enters such a basin, with loss, fit, and partial functions evolving together; (iii) low-rank structure (e.g. $2L$ spikes per layer L) yields a distinctive feature signature aligned with the hierarchical landscape. Within the stated scope (Section 9), understanding the loss landscape in these networks amounts to understanding how and when features are acquired. These observations support the SciForDL goal of building a scientific understanding of deep learning through reproducible experiments and explicit reporting of the relation between landscape geometry and learned representations.

A Appendix

Config files (e.g. `experiments/table/results_sumcos_selected_rerun/f3_N768_bs8_L2/config.json`) and the training script `experiments/table/run_selected_sumcos_configs.py` define the full parameterization and LR schedule. Broader baseline sweeps (sumcos rank 5/10, factor 1–5, batch 1–16) and their analysis (worked/failed counts, histograms of min loss per factor) are produced by `run_scaling_law_depth_width.py` (e.g. `--baseline-sweep`, `--baseline-sweep-rank5`) and `analyze_baseline_sweep.py` (`--sumcos`, `--sumcos-rank5`); batch-size analysis and run naming are documented in `experiments/table/BASELINE_SWEEP_SETUPS_AND_RANKS.md`. Log-ratio trajectory plots (Figures 4–5) and epochs-to-escape scaling plots (Figures **??**, **??**) come from `experiments/table/plateau/` (script `run_plateau_escape.py`); scaling law index and batch-size/rank sweep references: `experiments/table/INDEX_LOGRATIO_ESCAPE_SCALING.md`. Frequency/layer-scaling training and CSV summary by `--train` and `--analyze`. Stable baseline (cos $2\pi x$, $N = \text{width} = 1024$, rank 10, $L = 2$) reaches test error $\sim 1.9\times10^{-6}$ (momentum 0.7). Additional details are in `idea.md` and `experiments/partialfunctionanalysis/`.

B SCIENCE OF DL IMPROVEMENT CHALLENGE SUBMISSION

B.1 WHAT MODEL ARE YOU TARGETING?

Low-rank neural networks (MMNNs, RF-LR) for 1D regression on hierarchical Fourier targets (e.g. sum of cosines). We target the **relation between feature learning and the loss landscape**: how the geometry of the loss (plateaus, sharp drops, basins) connects to what the network learns internally (channel partials, frequency bands, oscillatory structure).

B.2 HOW DO YOUR RESULTS CONTRIBUTE—OR COULD POTENTIALLY CONTRIBUTE—TO UNDERSTANDING THESE MODELS?

We document that feature learning and the loss landscape are tightly linked: (i) each plateau/saddle corresponds to a frequency to be learned and dynamics are saddle-to-saddle; (ii) the landscape is a plateau with many sharp, deep basins that are hard to enter unless LR is small—at each LR divide the loss drops sharply (as in the provided figures) and the trajectory enters such a basin; (iii) observables such as mean minima per component and $2L$ spikes per layer L bridge landscape and feature learning. This contributes a concrete, empirical picture of how landscape geometry and learned representations relate in low-rank nets.

B.3 HOW DO YOU EXPECT YOUR SUBMISSION TO INFLUENCE FUTURE WORK?

The feature–landscape relation can be tested systematically (multiple seeds, architectures, metrics); scaling laws (plateau escape, batch size) and the link to feature-learning diagnostics (log-ratio, minima counts) can be refined; and the same methodology can be extended to other tasks and architectures to see how general the relation is.